PHP Full Course — Study Notes

Source: freeCodeCamp.org — "PHP Programming Language Tutorial - Full Course" (Mike Dane / Giraffe Academy)

---

1. Introduction

- PHP = server-side scripting language used to build dynamic, scalable websites.
- Integrates tightly with HTML — PHP code can be written directly alongside HTML.
- Runs **on the web server**, not in the browser, and interacts with clients (users).
- Course covers: setup, basics, HTML interaction, forms, general programming (if/loops/arrays), and OOP (classes/objects).

2. Windows Installation

1. Go to **php.net → Downloads → Windows downloads**.
2. Choose the version matching your OS (32-bit / 64-bit, thread-safe zip).
3. Extract the zip to `C:\php`.
4. Add `C:\php` to the **Windows PATH environment variable** (System Properties → Environment Variables → Path → New).
5. Verify in Command Prompt:
    - `echo %PATH%` → confirm PHP folder is listed.
    - `php -v` → should print the PHP version if installed correctly.

3. Choosing a Text Editor

- Any text editor that can save `.php` files works — no special config required.
- Options range from Notepad to specialized editors (syntax highlighting, error hints).
- Course uses **Atom** (by GitHub), but any editor is fine (VS Code, Sublime, etc.).

4. Hello World & Setup

- Start a built-in PHP dev server:

```
php -S localhost:4000
```

- This serves files from the **document root** (shown in terminal output).
- Create a `www` folder inside the document root; create `site.php` inside it.
- Basic PHP tag syntax:

```
<?php
    echo "Hello world";
```

```
?>
```

- Access via browser: `http://localhost:4000/www/site.php`.
- PHP code executes on the server; the result gets embedded into the HTML sent to the browser.

5. Writing HTML from PHP

- `echo` prints text (including full HTML tags) into the output document.
- Semicolons `;` end every PHP statement.
- Parentheses after `echo` are optional: `echo("text");` or `echo "text";`
- PHP instructions execute **top to bottom**, in the order written — reordering `echo` statements changes page output order.
- Example:

```
echo "<h1>Mike's Site</h1>";
echo "<hr>";
echo "<p>This is my site</p>";
```

6. Variables

- Declared with `$`: `$variableName = value;`
- Act as **containers** for reusable/changeable data — avoids manually editing repeated values everywhere.
- Strings need quotes; numbers do not.

```
$characterName = "John";
$characterAge = 35;
```

- Insert variables into strings directly: `"Once there was a man named $characterName"`.
- Variables can be reassigned at any point in the script — later reassignment changes value from that point onward.

7. Data Types

Type	Description	Example
String	Plain text, needs quotes	`"To be or not to be"`
Integer (int)	Whole number, no decimal	`30`, `-5`
Float / Double	Decimal number	`3.7`, `-8.47`
Boolean	true/false	`true`, `false`
Null	No value	`null`

- PHP distinguishes `30` (int) from `30.0` (float).
- Values don't have to be stored in variables — can be echoed directly.

8. Working with Strings

- Create/print: `echo "Graph Academy";`
- Common string functions:
  - `strtolower($str)` / `strtoupper($str)` — case conversion
  - `strlen($str)` — character count
  - `$str[0]` — access character by index (0-based)
  - `$str[0] = "B";` — modify a specific character
  - `str_replace($search, $replace, $str)` — replace substring
  - `substr($str, $start, $length)` — extract substring (length optional)
- Google "PHP string functions" for the full list — dozens exist.

9. Working with Numbers

- Arithmetic operators: `+` `-` `*` `/` `%` (modulus = remainder)
- Order of operations follows standard math rules; use `()` to override.
- Increment/decrement: `$num++`, `$num--`
- Compound assignment: `+=` `-=` `*=` `/=`
- Useful math functions:
  - `abs($n)` — absolute value
  - `pow($base, $exp)` — exponent
  - `sqrt($n)` — square root
  - `max($a, $b)` / `min($a, $b)` — largest/smallest
  - `round($n)`, `ceil($n)` (always rounds up), `floor($n)` (always rounds down)
- Many more available — search "PHP math functions".

10. Getting User Input

- HTML `<form>` bridges HTML and PHP:

```
<form action="site.php" method="get">
    Name: <input type="text" name="name">
    <input type="submit">
</form>
```

- Access submitted values in PHP with the **superglobal** `$_GET`:

```
echo $_GET["name"];
```

- Input `name` attribute must match the key used in `$_GET[...]`.

11. Building a Basic Calculator

- Two `<input type="number">` fields (`num1`, `num2`) + submit button.
- PHP adds them directly:

```
echo $_GET["num1"] + $_GET["num2"];
```

- With `method="get"`, submitted values appear in the URL as parameters (e.g. `?num1=10&num2=21`) — editable directly in the address bar.

12. Building a Mad Libs Game

- Multiple text inputs (`color`, `plural_noun`, `celebrity`) feed values into a story template using `$_GET`.
- Demonstrates combining form input + variables + string interpolation to build dynamic content.

13. URL Parameters

- Format: `page.php?key1=value1&key2=value2`
- `?` starts the query string; `&` separates multiple parameters.
- Useful for bookmarkable pages with state baked into the URL (e.g. Google search URLs).
- Downside: values are **fully visible and editable** by the user — not secure for sensitive data.

14. POST vs GET

	GET	**POST**
Data visibility	Shown in URL	Hidden from URL
Security	Low	Higher
Access in PHP	`$_GET[...]`	`$_POST[...]`
Typical use	Bookmarkable pages, search	Passwords, forms with sensitive data

- Change form `method="post"` and use `$_POST["field"]` to retrieve values.
- Most developers prefer **POST** for form submissions in general.

15. Arrays

- A container that stores **multiple** values (unlike a variable, which holds one).

```
$friends = array("Kevin", "Karen", "Oscar", "Mark", "Jim");
```

- Zero-indexed: `$friends[0]` → "Kevin"
- Modify an element: `$friends[1] = "Dwight";`
- Add new elements: `$friends[5] = "Angela";`
- Arrays can mix data types.
- `count($array)` — number of elements.

16. Using Checkboxes

- Checkbox group with matching `name="fruits[]"` (square brackets) stores checked values as an **array**.

```
<input type="checkbox" name="fruits[]" value="apples"> Apples
<input type="checkbox" name="fruits[]" value="oranges"> Oranges
```

- Retrieve in PHP:

```
$fruits = $_POST["fruits"];
echo $fruits[0]; // first checked value
```

17. Associative Arrays

- Store **key → value** pairs instead of numeric indexes:

```
$grades = array("Jim" => "A+", "Pam" => "B-", "Oscar" => "C+");
echo $grades["Jim"]; // A+
```

- Keys must be unique; values can repeat.
- Modify: `$grades["Jim"] = "F";`
- `count($grades)` still works.
- Useful for looking up dynamic data submitted by users, e.g. `$grades[$_POST["student"]]`.

18. Functions

- Groups reusable code into a named block:

```
function sayHi($name) {
    echo "Hello $name";
}
sayHi("Mike"); // must "call" the function to run it
```

- Parameters let you pass data in; a function can take multiple parameters.
- Code inside a function only runs when the function is **called**.

19. Return Statements

- `return` sends a value back to the caller and **immediately exits** the function (code after `return` never runs).

```
function cube($num) {
    return $num * $num * $num;
}
$result = cube(4); // 64
```

- Can return any data type: number, string, array, associative array, or nothing at all.

20. If Statements

- Basic decision-making structure:

```
if ($isMale) {
    echo "You are male";
} else {
    echo "You are not male";
}
```

- Logical operators:
  - && (AND) — both conditions must be true
  - || (OR) — at least one must be true
  - ! (NOT / negation) — flips true/false
- `elseif` lets you check additional conditions in sequence before the final `else`.

21. If Statements — Comparisons

- Comparison operators: >, <, >=, <=, == (equal), != (not equal)
- Comparisons resolve to a boolean and can be used directly as `if` conditions:

```
function getMax($num1, $num2) {
    if ($num1 > $num2) {
        return $num1;
    } else {
        return $num2;
    }
}
```

- Works for numbers and strings alike.

22. Building a Better Calculator

- Combines form input + `if/elseif/else` to branch on an **operator** string (`+`, `-`, `*`, `/`) submitted by the user.
- Falls into an `else` branch to handle invalid operator input ("Invalid operator").
- Note: HTML `<input type="number">` needs a `step` attribute (e.g. `step="0.01"`) to allow decimal input — this is an HTML limitation, not PHP.

23. Switch Statements

- Cleaner alternative to a long if/elseif chain when comparing **one value** against many possibilities:

```
switch ($grade) {
    case "A":
        echo "You did amazing";
        break;
    case "B":
        echo "You did pretty good";
        break;
    default:
        echo "Invalid grade";
}
```

- `break` exits the switch after a matching case (omitting it causes **fall-through** into the next case).
- `default` handles any value not matched by a case.

24. While Loops

- Repeats code **while a condition remains true**:

```
$index = 1;
while ($index <= 5) {
    echo $index;
    $index++;
}
```

- Condition is checked **before** each iteration.
- **Infinite loop** warning: forgetting to update the loop variable causes the condition to stay true forever — can crash/slow the browser.
- **do-while loop**: executes the loop body **once before** checking the condition:

```
do {
    echo $index;
} while ($index <= 5);
```

25. For Loops

- Combines initialization, condition, and increment into one line — ideal when tracking an iterating variable:

```
for ($i = 1; $i <= 5; $i++) {
    echo $i;
}
```

- Very common for looping through arrays:

```
$luckyNumbers = array(4, 8, 15, 16, 23, 42);
for ($i = 0; $i < count($luckyNumbers); $i++) {
    echo $luckyNumbers[$i];
}
```

(Start at 0 since array indexes are zero-based; use < not <= since the last index is `count - 1`.)

26. Comments

- Single-line: `// comment`
- Multi-line block: `/* comment spanning multiple lines */`
- Comments are ignored by PHP — used for notes, documentation, or **"commenting out"** code temporarily for testing without deleting it.

27. Including HTML (`include`)

- Pulls the contents of another file into the current file — useful for reusable header/footer templates:

```
include "header.html";
... page content ...
include "footer.html";
```

- Update the header/footer file once, and the change reflects across every page that includes it — promotes modular website design.

28. Include: PHP Files

- `include` also works with `.php` files, not just HTML.
- Enables **templates with variables**: define an "empty" template file with variables (e.g. `$title`, `$author`, `$wordCount`), then assign values to those variables *before* the `include` statement in the file that uses it.
- Can also include files containing reusable **functions and variables** (e.g. a `useful-tools.php` utility file) to use that functionality anywhere it's included.

29. Classes & Objects

- A **class** is a blueprint/custom data type; an **object** is an instance of that class.

```
class Book {
    var $title;
    var $author;
    var $pages;
}

$book1 = new Book();
$book1->title = "Harry Potter";
$book1->author = "JK Rowling";
$book1->pages = 400;
```

- Access attributes with `->`.
- Multiple objects of the same class can hold independent data (e.g. `$book2` for a different book).

30. Constructors

- Special method `__construct()` runs automatically whenever a new object is created —

used to set initial attribute values in one step:

```
class Book {
    var $title;
    var $author;
    var $pages;

    function __construct($title, $author, $pages) {
        $this->title = $title;
        $this->author = $author;
        $this->pages = $pages;
    }
}

$book1 = new Book("Harry Potter", "JK Rowling", 400);
```

- `$this` refers to the current object being created/used.
- Drastically reduces the code needed to create fully-populated objects.

31. Object Functions (Methods)

- Functions defined inside a class that operate on that object's own data:

```
class Student {
    var $name;
    var $major;
    var $gpa;

    function __construct($name, $major, $gpa) {
        $this->name = $name;
        $this->major = $major;
        $this->gpa = $gpa;
    }

    function hasHonors() {
        if ($this->gpa >= 3.5) {
            return "true";
        } else {
            return "false";
        }
    }
}

$student1->hasHonors(); // uses that specific object's GPA
```

- Write once, reuse across every object of that class; `$this` always refers to the calling object's own data.

32. Getters & Setters

- **Visibility modifiers** control access to attributes:
    - `public` — accessible from anywhere.
    - `private` — accessible only from inside the class itself.
- Pattern for controlled/validated access:

```
class Movie {
    private $rating;
```

```
    function __construct($title, $rating) {
        $this->title = $title;
        $this->setRating($rating);
    }

    function getRating() {
        return $this->rating;
    }

    function setRating($rating) {
        if ($rating == "G" || $rating == "PG" || $rating == "PG-13" ||
            $rating == "R" || $rating == "NR") {
            $this->rating = $rating;
        } else {
            $this->rating = "NR";
        }
    }
}
```

- Getters/setters let you validate or restrict what values can be assigned (e.g. only allow valid movie ratings), enforcing rules even through the constructor.

33. Inheritance

- A class can **extend** another class to inherit all its attributes/methods:

```
class Chef {
    function makeChicken() { echo "The chef makes chicken"; }
    function makeSalad()  { echo "The chef makes salad"; }
    function makeSpecialDish() { echo "The chef makes bbq ribs"; }
}

class ItalianChef extends Chef {
    function makePasta() { echo "The chef makes pasta"; }

    // Overriding an inherited method:
    function makeSpecialDish() { echo "The chef makes chicken parm"; }
}
```

- `ItalianChef` automatically gets `makeChicken()` and `makeSalad()` from `Chef`, plus its own `makePasta()`.
- **Overriding**: redefining a parent method inside the child class replaces that behavior only for the child.
- Use inheritance when one class is a more specialized version of another ("is-a" relationship) to avoid duplicating code.

Quick Reference: Superglobals & Key Syntax

- `$_GET["key"]` — retrieve form data sent via GET (also appears in URL)
- `$_POST["key"]` — retrieve form data sent via POST (hidden from URL)
- `array()` or `[...]` — create an array

- `array("key" => "value", ...)` — associative array
- `function name($params) { ... return ...; }` — define/return from a function
- `class Name { ... }` / `new Name()` — define/instantiate a class
- `$this->attribute` — access current object's own attribute inside a class
- `ClassName extends ParentClass` — inheritance